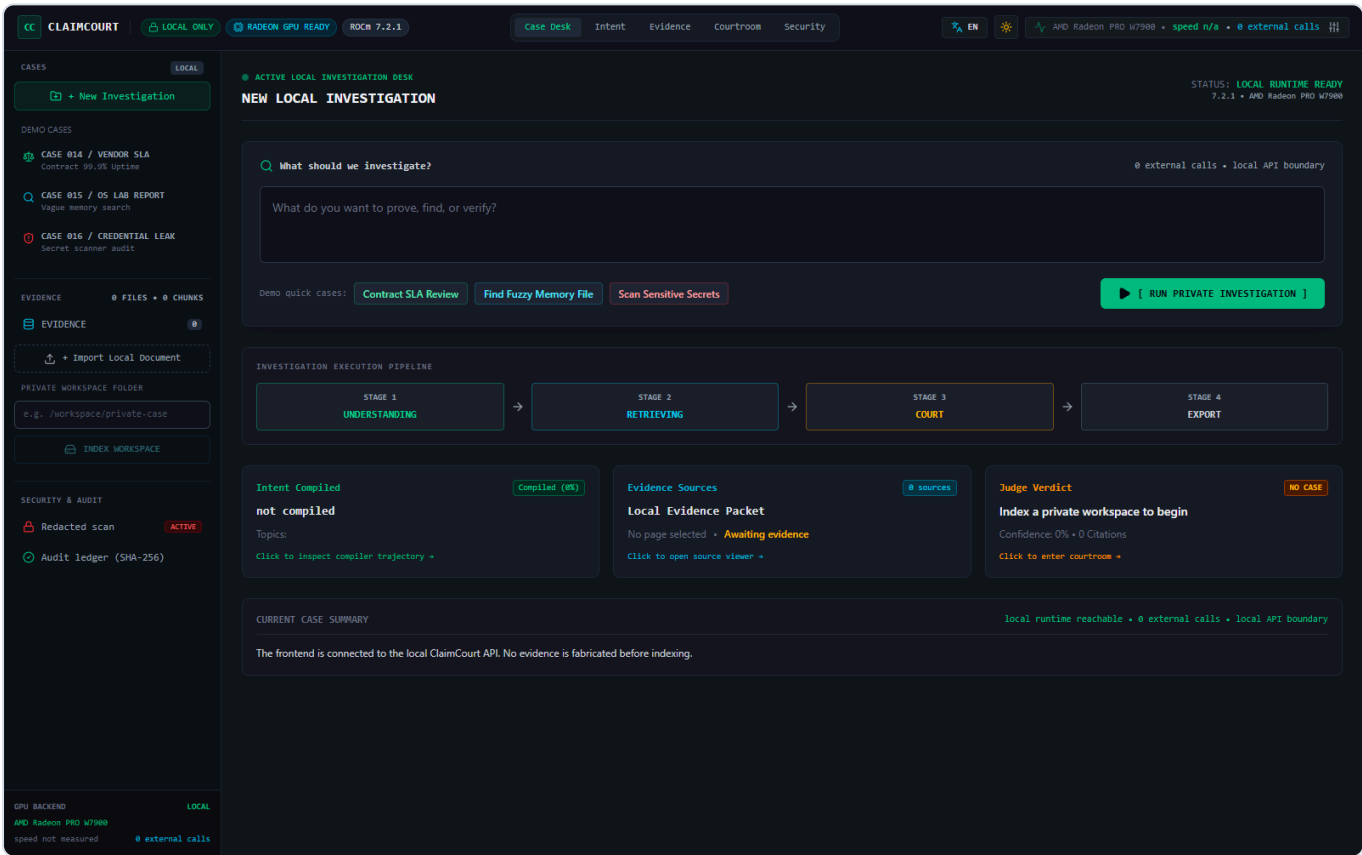


CLAIMCOURT / TEAM ZI FEI YU

ClaimCourt

A Private Local AI Evidence Court

AMD AI DevMaster Hackathon 2026
Track 2 - Development & Local Deployment of Private AI Agents



Private documents stay local. Evidence is retrieved, challenged, cited, and exported only after approval.

Team: Zi Fei Yu

Track: Track 2 - Development & Local Deployment of Private AI Agents

Competition: AMD AI DevMaster Hackathon 2026

Executive Summary

ClaimCourt is a private, evidence-grounded local AI agent for people who remember the meaning of a record but not its filename, and for teams that must resolve disputed commitments across contracts, email, meeting notes, presentations, and internal reports.

Its first innovation is the **Fuzzy Intent Compiler**. Instead of sending a vague sentence directly to a vector database, ClaimCourt compiles remembered topics, artifact types, dates, entities, causal relations, exclusions, file roles, and clarification requirements into a structured local search plan. The plan drives a hybrid retrieval pipeline that combines BM25, SQLite FTS5, word and character TF-IDF, BGE embeddings, metadata, file-family grouping, and a local cross-encoder reranker.

Its second innovation is the **Evidence Court**. A single local instruction model is called through three controlled stages: prosecution presents supporting evidence, defense searches for counter-evidence and exceptions, and judge issues a schema-valid verdict using only citations from the retrieved packet. The output includes a verdict, confidence, contradictions, timeline, missing evidence, and recommended next action.

The verified competition stack runs entirely on an AMD Radeon PRO W7900-class GPU with ROCm. It uses Qwen3-14B BF16 through vLLM, a ClaimCourt intent LoRA, BAAI/bge-small-zh-v1.5 through the vLLM pooling runner, and a local BGE cross-encoder. No closed remote model API is required.

Application Scenarios

1. Disputed commitment review

A customer says sales promised 99.9% uptime. Procurement sees no SLA in the signed agreement. Engineering remembers a conditional Q4 target. ClaimCourt retrieves all relevant records, distinguishes drafts and marketing language from signed obligations, and produces a cited decision brief.

2. Fuzzy private file recovery

A user asks: "Find the presentation I wrote about the customer's delayed launch and Q4 risk." ClaimCourt compiles the recollection into a transparent intent plan, groups draft and final versions, selects the authoritative file, and generates a cited summary from the complete artifact.

3. Redacted sensitive-record location

A user remembers that a local file contains a private key or credential. ClaimCourt scans only the selected workspace and returns the file location, category, fingerprint, and redacted preview. It does not reveal the secret value or send it to the language model.

4. Approval and audit reconstruction

A team asks who approved a requirement change and under what conditions. ClaimCourt reconstructs dated evidence from meeting notes, approval logs, email, and change requests, then identifies contradictions and missing approval records.

Agent Architecture

```
User-selected private workspace
|
v
Local parsers: PDF / DOC / DOCX / PPTX / TXT / MD / EML / OCR
|
v
Parent-child chunks + SHA-256 evidence ledger + durable FTS5
|
v
Fuzzy Intent Compiler (Qwen3-14B + ClaimCourt LoRA)
|
v
Hybrid retrieval: BM25 + FTS5 + TF-IDF + BGE + metadata
|
v
File-family authority rules + local BGE cross-encoder reranker
|
v
Retrieved evidence packet with allowed citation IDs
|
+--> Prosecutor: strongest supporting evidence
+--> Defense: counter-evidence, exceptions, contradictions
+--> Judge: supported / contradicted / insufficient_evidence
|
v
Cited verdict + timeline + missing evidence + next action
|
v
Explicit user approval --> local Markdown decision brief
```

The three roles are controlled serial stages using one local model. ClaimCourt does not claim that three autonomous models collaborate. This design reduces VRAM use, keeps behavior auditable, and makes every transition visible.

Core Capabilities

Local RAG with production-oriented retrieval

- Structure-aware parent-child chunking with page, slide, and document locators.
- BM25, SQLite FTS5, word TF-IDF, character n-gram, BGE semantic similarity, metadata, and filename evidence.
- Local cross-encoder reranking over a bounded shortlist.
- File-family grouping for drafts, signed copies, exports, backups, and final versions.
- Deterministic authority rules prevent an embedding score from selecting a draft over a final or signed file unless the user explicitly requests the draft.
- OCR diagnostics and legacy DOC parsing failures are visible instead of silently ignored.

Tool calling and routing

One private request is routed to an evidence court, fuzzy file locator, artifact summarizer, workspace indexer, or redacted sensitive-record scanner. The local intent plan is inspectable and persisted for audit.

Multi-step planning

ClaimCourt executes a constrained workflow: understand intent, retrieve evidence, prosecute, defend, judge, validate citations, and request approval before export. Model output is validated against strict JSON contracts at every role boundary.

Local memory

The workspace index persists source hashes, evidence locators, prior versions, query plans, result identities, confidence, reranker status, and failure diagnostics. Changed files are re-indexed and archived evidence is

retained for audit.

Privacy and permission controls

- Model endpoints are loopback-only by default.
- Documents and prompts remain inside the selected host.
- Raw secrets are redacted before model, UI, history, or export paths.
- Unknown model citations are removed and recorded as quality failures.
- Real workspaces fail closed when the local model is unavailable.
- Report generation requires explicit user approval.

Model and Local Deployment

Verified Radeon stack

Layer	Verified component
GPU	AMD Radeon PRO W7900 class, gfx1100, 51.52 GB VRAM
Runtime	ROCm + vLLM, OpenAI-compatible local endpoints
Judge and role model	Qwen3-14B BF16
Intent specialization	ClaimCourt LoRA trained on synthetic query-to-intent plans
Embedding	BAAI/bge-small-zh-v1.5, 512 dimensions, vLLM pooling runner
Reranker	Local bge-reranker-base cross-encoder
UI	React/Vite interface backed by a local Python API

The judge and LoRA router are served at `http://127.0.0.1:8000/v1`. The embedding model is served at `http://127.0.0.1:8001/v1`. The browser connects only to the local ClaimCourt API.

Startup

```
cd /workspace/claimcourt
python -m pip install -r requirements-radeon.txt
bash scripts/start_radeon_stack.sh
```

The startup script verifies the judge endpoint, embedding endpoint, and local API health. The repository also contains a deterministic synthetic fallback for UI development, but championship mode rejects fallback and requires the live local model.

AMD Radeon Optimization

1. **One shared judge service:** prosecution, defense, and judge reuse one Qwen3-14B process instead of loading three model copies.
2. **BF16 on W7900:** the verified model uses BF16 and fits alongside the embedding endpoint and API stack.
3. **Controlled VRAM allocation:** judge `gpu-memory-utilization=0.82`; embedding pooling runner `0.10`; observed full-stack peak approximately 45.3 GB of 51.52 GB.
4. **Thinking disabled for schema calls:** `enable_thinking=false` avoids an unnecessary reasoning stream and protects JSON output contracts.
5. **Bounded context:** the judge uses an 8,192-token context and receives only a retrieved evidence packet, not the complete workspace.

- 6. **Small dedicated embedding runner:** BGE-small provides 512-dimensional local vectors with a 512-token input budget.
- 7. **Shortlist reranking:** the cross-encoder scores only hybrid-retrieval candidates instead of the full corpus.
- 8. **Concurrent intent evaluation:** the live holdout used router concurrency 8 and reached 100% GPU utilization during the routing phase.

Measured Results

Live Radeon court gate

Metric	Result
First-token latency	0.27 s
Generation throughput	14.3 tokens/s
Completion tokens	1,778
Complete three-role latency	126.44 s
Court mode	local vLLM
Expected verdict	insufficient_evidence

Live fuzzy-intent holdout

Metric	Result
Frozen queries	320
Overall pass rate	100%
Intent contract accuracy	100%
Single-target Top-1	100%
Multi-target full coverage	100%
No-match accuracy	100%
Clarification accuracy	100%
Unsafe wrong auto-selections	0
Live Router / embedding / reranker	all passed

The holdout is synthetic and frozen by SHA-256. It is an engineering regression gate, not a claim of universal retrieval quality. A separate private directory evaluation is kept outside the public repository and only sanitized aggregate metrics are published.

Demo Flow

1. Show rocminfo, Radeon GPU status, vLLM model endpoints, and the ClaimCourt local runtime panel.
2. Load the synthetic private workspace containing contracts, email, meeting notes, approvals, drafts, and unrelated files.

3. Ask: "Did the vendor contractually commit to 99.9% uptime?"
4. Inspect the compiled intent, evidence sources, prosecution, defense, cited verdict, contradictions, and timeline.
5. Approve the local decision brief and show that no export occurs before approval.
6. Display first-token latency, throughput, VRAM, model, ROCm, and zero-external-call evidence.

Repository Verification

```
python -m unittest discover -s tests -t .  
python scripts/championship_check.py --output championship_results.local.json  
python scripts/adversarial_holdout_check.py
```

The current release passes 78 Python tests. Radeon live evidence is stored in docs/evidence/radeon-championship-live-20260805.json and docs/evidence/radeon-fuzzy-holdout-live-20260805.json.

Limitations and Honest Scope

ClaimCourt is a private alpha and not legal advice. The public benchmark is synthetic. Complete court latency is still high for repeated interactive use. Encryption at rest, multi-user authentication, workspace isolation, and long-term owner dogfood remain future production work. The system surfaces these limits rather than claiming production readiness.

ClaimCourt / Team Zi Fei Yu / AMD AI DevMaster Hackathon 2026 / Track 2